CPU Scheduling Algorithms

1 First-Come, First-Served (FCFS)

The simplest non-preemptive CPU scheduling algorithm. Processes are allocated to the CPU in the exact order of their arrival. It is easy to implement but can suffer from the "convoy effect."

Input & Output Format

Input: Integer `n`. Next `n` lines: `Arrival Time (AT)` and `Burst Time (BT)`.
Output: Table of PID, AT, BT, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```cpp
#include<bits/stdc++.h>
using namespace std;
#define nl cout << endl;

struct Process{ int pid, at, bt, ct, tat, wt;
  void show(){ cout << pid <<"\t" << at <<"\t" << bt <<"\t" << ct <<"\t" << tat <<"\t" << wt << endl; }
};
struct Gantt{ int pid, start, end; };

void showGanttChart(vector<Gantt> g){
  cout << "\n\nGantt Chart\n\n";
  for(auto x : g) cout <<"| P" << x.pid <<" ";
  cout << "|\n" << g[0].start;
  for(auto x : g) cout << setw(5) << x.end;
  nl;
}

int32_t main(){
  int n = 0; cin >> n;
  vector<Process> p(n);
  for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt; }

  sort(p.begin(), p.end(), [](Process a, Process b){
    return a.at == b.at ? a.pid < b.pid : a.at < b.at;
  });

  int curTime = 0; vector<Gantt> g;
  for(int i = 0; i < n; i++){
    if(curTime < p[i].at) curTime = p[i].at;
    curTime += p[i].bt;
    g.push_back({p[i].pid, curTime - p[i].bt, curTime});
    p[i].ct = curTime; p[i].tat = p[i].ct - p[i].at; p[i].wt = p[i].tat - p[i].bt;
  }

  cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
  double totTat = 0, totWt = 0;
  for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
  cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
  showGanttChart(g);
}
```

---

2 Shortest Job First (SJF) - Non-Preemptive

A non-preemptive algorithm that selects the waiting process with the smallest execution time (Burst Time). Resolves ties using arrival time.

Input & Output Format

Input: Integer `n`. Next `n` lines: `Arrival Time (AT)` and `Burst Time (BT)`.
Output: Table of PID, AT, BT, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```cpp
#include<bits/stdc++.h>
using namespace std;
#define nl cout << endl;

struct Process{ int pid, at, bt, ct, tat, wt; bool finished = false;
  void show(){ cout << pid <<"\t" << at <<"\t" << bt <<"\t" << ct <<"\t" << tat <<"\t" << wt << endl; }
```

```
};
struct Gantt{ int pid, start, end; };

void showGanttChart(vector<Gantt> g){
  cout << "\n\nGantt Chart\n\n";
  for(auto x : g) cout <<"| P" << x.pid <<" ";
  cout << "|\n" << g[0].start;
  for(auto x : g) cout << setw(5) << x.end;
  nl;
}

int32_t main(){
  int n = 0; cin >> n;
  vector<Process> p(n); vector<Gantt> g;
  for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt; }

  int completed = 0, curTime = 0;
  while(completed < n){
    int selected = -1;
    for(int i = 0; i < n; i++){
      if(!p[i].finished and p[i].at <= curTime){
        if(selected == -1 or p[i].bt < p[selected].bt or (p[i].bt == p[selected].bt and p[i].at < p[
    selected].at)) selected = i;
      }
    }
    if(selected == -1){ curTime++; continue; }

    curTime += p[selected].bt;
    g.push_back({p[selected].pid, curTime - p[selected].bt, curTime});
    p[selected].ct = curTime; p[selected].tat = p[selected].ct - p[selected].at; p[selected].wt = p[
    selected].tat - p[selected].bt;
    p[selected].finished = true; completed++;
  }

  cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
  double totTat = 0, totWt = 0;
  for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
  cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
  showGanttChart(g);
}
```

3 Round Robin (RR)

A preemptive scheduling algorithm where each process is assigned a fixed time slot called a "Time Quantum." If it doesn't finish, it moves to the back of the queue.

Input & Output Format

Input: Integer `n`. Next `n` lines: `AT` & `BT`. Next line: `Time Quantum (tq)`.
Output: Table of PID, AT, BT, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```cpp
#include<bits/stdc++.h>
using namespace std;
#define nl cout << endl;

struct Process{ int pid, at, bt, ct, tat, wt;
  void show(){ cout << pid <<"\t" << at <<"\t" << bt <<"\t" << ct <<"\t" << tat <<"\t" << wt << endl; }
};
struct Gantt{ int pid, start, end; };

void showGanttChart(vector<Gantt> g){
  cout << "\n\nGantt Chart\n\n";
  for(auto x : g) cout <<"| P" << x.pid <<" ";
  cout << "|\n" << g[0].start;
  for(auto x : g) cout << setw(5) << x.end;
  nl;
}

int32_t main(){
  int n = 0; cin >> n;
  vector<Gantt> g; vector<Process> p(n);
  for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt; }

  int tq = 0; cin >> tq;
  vector<int> rem(n); for(int i = 0; i < n; i++) rem[i] = p[i].bt;

  sort(p.begin(), p.end(), [](Process a, Process b){ return a.at == b.at ? a.pid < b.pid : a.at < b.at; });

  int curTime = 0, completed = 0, i = 0; queue<int> q;
  while(completed < n){
    if(q.empty() and i < n and curTime < p[i].at) curTime = p[i].at;
    while(i < n and p[i].at <= curTime){ q.push(i); i++; }

    int idx = q.front(); q.pop();
    int excTime = min(tq, rem[idx]);
    rem[idx] -= excTime;
    g.push_back({p[idx].pid, curTime, curTime + excTime});
    curTime += excTime;

    while(i < n and p[i].at <= curTime){ q.push(i); i++; }

    if(rem[idx]){ q.push(idx); }
    else{
      completed++;
      p[idx].ct = curTime; p[idx].tat = p[idx].ct - p[idx].at; p[idx].wt = p[idx].tat - p[idx].bt;
    }
  }

  cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
  double totTat = 0, totWt = 0;
  for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
  cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
  showGanttChart(g);
}
```

---

4 Priority Scheduling (Non-Preemptive)

The CPU is allocated to the process with the highest priority (smaller integer indicates a higher priority). Once a process starts, it runs to completion.

Input & Output Format

Input: Integer `n`. Next `n` lines: `AT`, `BT`, & `Priority`.
Output: Table of PID, AT, BT, Priority, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```cpp
#include<bits/stdc++.h>
using namespace std;
#define nl cout << endl;

struct Process{ int pid, at, bt, priority, ct, tat, wt; bool finished = false;
  void show(){ cout << pid <<"\t" << at <<"\t" << bt <<"\t" << priority <<"\t\t\t" << ct <<"\t" <<
    tat <<"\t" << wt << endl; }
};
struct Gantt{ int pid, start, end; };

void showGanttChart(vector<Gantt> g){
  cout << "\n\nGantt Chart\n\n";
  for(auto x : g) cout <<"| P" << x.pid <<" ";
  cout << "|\n" << g[0].start;
  for(auto x : g) cout << setw(5) << x.end;
  nl;
}

int32_t main(){
  int n = 0; cin >> n;
  vector<Process> p(n); vector<Gantt> g;
  for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt >> p[i].priority; }

  int curTime = 0, completed = 0;
  while(completed < n){
    int selected = -1;
    for(int i = 0; i < n; i++){
      if(!p[i].finished and p[i].at <= curTime){
        if(selected == -1 or p[i].priority < p[selected].priority) selected = i;
      }
    }
    if(selected == -1){ curTime++; continue; }

    g.push_back({p[selected].pid, curTime, curTime + p[selected].bt});
    curTime += p[selected].bt;
    p[selected].ct = curTime; p[selected].tat = p[selected].ct - p[selected].at; p[selected].wt = p[
    selected].tat - p[selected].bt;
    p[selected].finished = true; completed++;
  }

  cout << "\nPID\tAT\tBT\tPriority\tCT\tTAT\tWT\n";
  double totTat = 0, totWt = 0;
  for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
  cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
  showGanttChart(g);
}
```

5 Shortest Remaining Time First (SRTF)

The preemptive version of SJF. It schedules the process with the least remaining burst time. Preempts currently running process if a shorter job arrives.

Input & Output Format

Input: Integer `n`. Next `n` lines: `AT` & `BT`.
Output: Table of PID, AT, BT, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```cpp
#include<bits/stdc++.h>
using namespace std;
#define nl cout << endl;

struct Process{ int pid, at, bt, ct, tat, wt;
  void show(){ cout << pid <<"\t" << at <<"\t" << bt <<"\t" << ct <<"\t" << tat <<"\t" << wt << endl; }
};
struct Gantt{ int pid, start, end; };

void showGanttChart(vector<Gantt> g){
  cout << "\n\nGantt Chart\n\n";
  for(auto x : g) cout <<"| P" << x.pid <<" ";
  cout << "|\n" << g[0].start;
  for(auto x : g) cout << setw(5) << x.end;
  nl;
}

int32_t main(){
  int n = 0; cin >> n;
  vector<Gantt> g; vector<Process> p(n); vector<int> rem(n);
  for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt; rem[i] = p[i].bt; }

  int curTime = 0, completed = 0;
  while(completed < n){
    int selected = -1;
    for(int i = 0; i < n; i++){
      if(p[i].at <= curTime and rem[i]){
        if(selected == -1 or rem[i] < rem[selected] or (rem[i] == rem[selected] and p[i].at < p[selected].at)) selected = i;
      }
    }
    if(selected == -1){ curTime++; continue; }
    rem[selected]--;

    if(!g.empty() && g.back().pid == p[selected].pid) g.back().end = curTime + 1;
    else g.push_back({p[selected].pid, curTime, curTime + 1});

    curTime++;
    if(rem[selected] == 0){
      p[selected].ct = curTime; p[selected].tat = p[selected].ct - p[selected].at; p[selected].wt = p[selected].tat - p[selected].bt;
      completed++;
    }
  }

  cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
  double totTat = 0, totWt = 0;
  for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
  cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
  showGanttChart(g);
}
```

---

6 Priority Scheduling (Preemptive)

If a new process arrives in the ready queue with a higher priority (smaller number) than the currently executing process, the CPU is immediately given to the new process.

Input & Output Format

Input: Integer `n`. Next `n` lines: `AT`, `BT`, & `Priority`.
Output: Table of PID, AT, BT, Priority, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```cpp
#include<bits/stdc++.h>
using namespace std;
#define nl cout << endl;

struct Process{ int pid, at, bt, priority, ct, tat, wt, rem_bt; bool finished = false;
  void show(){ cout << pid <<"\t" << at <<"\t" << bt <<"\t" << priority <<"\t\t\t" << ct <<"\t" <<
    tat <<"\t" << wt << endl; }
};
struct Gantt{ int pid, start, end; };

void showGanttChart(vector<Gantt> g){
  cout << "\n\nGantt Chart\n\n";
  for(auto x : g) cout <<"| P" << x.pid <<" ";
  cout << "|\n" << g[0].start;
  for(auto x : g) cout << setw(5) << x.end;
  nl;
}

int32_t main(){
  int n = 0; cin >> n;
  vector<Process> p(n); vector<Gantt> g;
  for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt >> p[i].priority; p[i].
    rem_bt = p[i].bt; }

  int curTime = 0, completed = 0;
  while(completed < n){
    int selected = -1;
    for(int i = 0; i < n; i++){
      if(!p[i].finished and p[i].at <= curTime){
        if(selected == -1 or p[i].priority < p[selected].priority) selected = i;
      }
    }
    if(selected == -1){ curTime++; continue; }

    curTime++; p[selected].rem_bt--;

    if(!g.empty() and g.back().pid == selected + 1) g.back().end = curTime;
    else g.push_back({selected + 1, curTime-1, curTime});

    if(p[selected].rem_bt == 0){
      p[selected].ct = curTime; p[selected].tat = p[selected].ct - p[selected].at; p[selected].wt =
    p[selected].tat - p[selected].bt;
      p[selected].finished = true; completed++;
    }
  }

  cout << "\nPID\tAT\tBT\tPriority\tCT\tTAT\tWT\n";
  double totTat = 0, totWt = 0;
  for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
  cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
  showGanttChart(g);
}
```

7 Multilevel Queue Scheduling

Ready queue is partitioned. Queue 1 processes run FCFS, and Queue 2 processes run Round Robin (Time Quantum = 2). Queue 1 has absolute priority over Queue 2.

Input & Output Format

Input: Integer `n`. Next `n` lines: `AT`, `BT`, & `Queue Number (1 or 2)`.
Output: Table of PID, AT, BT, Queue, CT, TAT, WT; Average TAT & WT; Gant Chart (with Idle).

```cpp
#include<bits/stdc++.h>
using namespace std;
#define nl cout << endl;

struct Process{ int pid, at, bt, priority, ct, tat, wt, queue_no, rem_bt; bool finished = false, vis = false;
  void show(){ cout << pid <<"\t" << at <<"\t" << bt <<"\t" << priority <<"\t\t\t" << ct <<"\t" << tat <<"\t" << wt << endl; }
};
struct Gantt{ int pid, start, end; };

void showGanttChart(vector<Gantt> g){
  cout << "\n\nGantt Chart\n\n";
  for(auto x : g){ if(x.pid == -1) cout <<"|Idle"; else cout <<"| P" << x.pid <<" "; }
  cout << "|\n" << g[0].start;
  for(auto x : g) cout << setw(5) << x.end;
  nl;
}

int32_t main(){
  int n = 0; cin >> n;
  vector<Process> p(n); vector<Gantt> g;
  for(int i = 0; i < n; i++){
    p[i].pid = i + 1; cin >> p[i].at >> p[i].bt >> p[i].queue_no;
    p[i].rem_bt = p[i].bt; p[i].priority = p[i].queue_no;
  }

  queue<int> q1, q2; int tq = 2, curTime = 0, completed = 0;
  auto doneTask = [&](int i){
    p[i].ct = curTime; p[i].tat = p[i].ct - p[i].at; p[i].wt = p[i].tat - p[i].bt; p[i].finished = true; completed++;
  };

  while(completed < n){
    for(int i = 0; i < n; i++){
      if(!p[i].finished and p[i].at <= curTime and !p[i].vis){
        if(p[i].queue_no == 1) q1.push(i); else q2.push(i);
        p[i].vis = true;
      }
    }
    if(!q1.empty()){
      int i = q1.front(); q1.pop();
      g.push_back({p[i].pid, curTime, curTime + p[i].bt});
      curTime += p[i].bt; doneTask(i);
    }else if(!q2.empty()){
      int i = q2.front(); q2.pop();
      int runTime = min(tq, p[i].rem_bt);
      g.push_back({p[i].pid, curTime, curTime + runTime});
      curTime += runTime; p[i].rem_bt -= runTime;
      if(p[i].rem_bt == 0) doneTask(i); else q2.push(i);
    }else{
      g.push_back({-1, curTime, curTime + 1}); curTime++;
    }
  }

  cout << "\nPID\tAT\tBT\tQueue\t\tCT\tTAT\tWT\n";
  double totTat = 0, totWt = 0;
  for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
  cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
  showGanttChart(g);
}
```

8 Multilevel Feedback Queue (MLFQ)

A dynamic queue system. Queue 1: RR (tq=2) → Queue 2: RR (tq=4) → Queue 3: SJF. Processes demote if they exhaust their time quantum.

Input & Output Format

Input: Integer `n`. Next `n` lines: `AT` & `BT`.
Output: Table of PID, AT, BT, CT, TAT, WT; Average TAT & WT; Gantt Chart.

```cpp
#include<bits/stdc++.h>
using namespace std;
#define nl cout << endl;

struct Process{ int pid, at, bt, priority, ct, tat, wt, queue_no, rem_bt; bool finished = false, vis = false;
  void show(){ cout << pid <<"\t" << at <<"\t" << bt <<"\t" << ct <<"\t" << tat <<"\t" << wt << endl; }
};
struct Gantt{ int pid, start, end; };

void showGanttChart(vector<Gantt> g){
  cout << "\n\nGantt Chart\n\n";
  for(auto x : g){ if(x.pid == -1) cout <<"|Idle"; else cout <<"| P" << x.pid <<" "; }
  cout << "|\n" << g[0].start;
  for(auto x : g) cout << setw(5) << x.end;
  nl;
}

int32_t main(){
  int n = 0; cin >> n;
  vector<Process> p(n); vector<Gantt> g;
  for(int i = 0; i < n; i++){ p[i].pid = i + 1; cin >> p[i].at >> p[i].bt; p[i].rem_bt = p[i].bt; }

  queue<int> q1, q2, q3; int tq1 = 2, tq2 = 4, curTime = 0, completed = 0;

  auto doneTask = [&](int i){
    p[i].ct = curTime; p[i].tat = p[i].ct - p[i].at; p[i].wt = p[i].tat - p[i].bt; p[i].finished = true; completed++;
  };

  while(completed < n){
    for(int i = 0; i < n; i++){
      if(!p[i].finished and p[i].at <= curTime and !p[i].vis){ q1.push(i); p[i].vis = true; }
    }

    if(!q1.empty()){
      int i = q1.front(); q1.pop();
      int runTime = min(tq1, p[i].rem_bt);
      g.push_back({p[i].pid, curTime, curTime + runTime});
      curTime += runTime; p[i].rem_bt -= runTime;
      if(p[i].rem_bt == 0) doneTask(i); else q2.push(i);
    }else if(!q2.empty()){
      int i = q2.front(); q2.pop();
      int runTime = min(tq2, p[i].rem_bt);
      g.push_back({p[i].pid, curTime, curTime + runTime});
      curTime += runTime; p[i].rem_bt -= runTime;
      if(p[i].rem_bt == 0) doneTask(i); else q3.push(i);
    }else if(!q3.empty()){
      int i = q3.front(); q3.pop();
      g.push_back({p[i].pid, curTime, curTime + p[i].rem_bt});
      curTime += p[i].rem_bt; p[i].rem_bt = 0; doneTask(i);
    }else {
      g.push_back({-1, curTime, curTime + 1}); curTime++;
    }
  }

  cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
  double totTat = 0, totWt = 0;
  for(auto i : p){ totTat += i.tat; totWt += i.wt; i.show(); }
  cout << "Total turnaround time: " << totTat / n << "\nTotal waiting time: " << totWt / n << endl;
  showGanttChart(g);
}
```